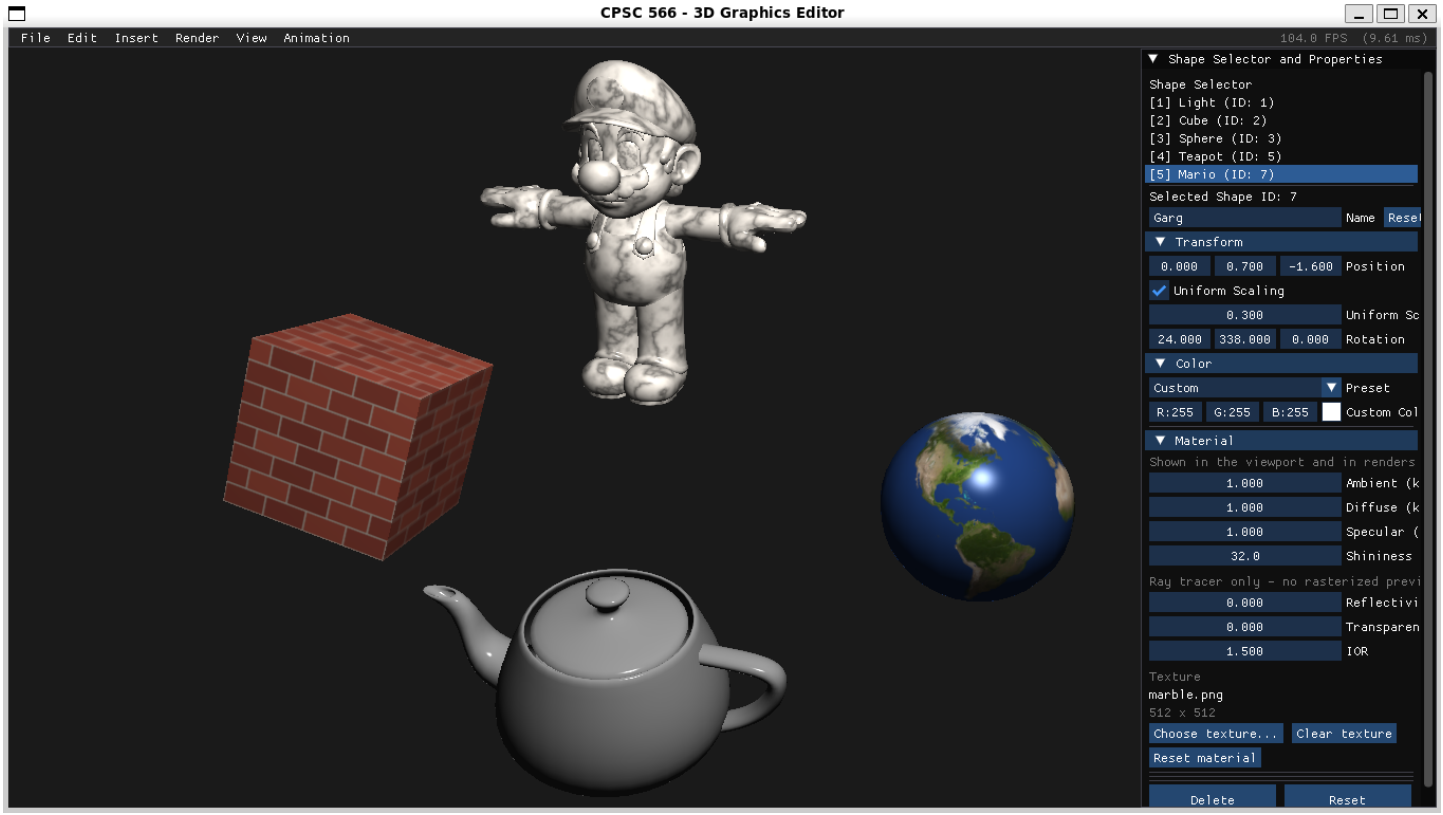
CALIFORNIA STATE UNIVERSITY FULLERTON

Assignment 5

CPSC 484 – Principles of Computer Graphics
Assignment 5: Textures and Materials
Fall 2026
Professor: Dave Garcia-Gomez
Version: 2026-10-27



1 Getting Started

In this assignment, you will extend the 3D Model Scene Editor by implementing texture sampling, texture-coordinate generation, and Phong lighting. These features give surfaces image detail and determine how light interacts with their materials.

A texture determines the color associated with a point on a surface. Lighting determines how much of that color reaches the viewer. Both are necessary for a convincing result: texture without lighting appears like an image attached to a surface, while lighting applied to a flat color does not provide surface detail.

CALIFORNIA STATE UNIVERSITY FULLERTON

Assignment 5

The texture pipeline involves two distinct operations. First, texture coordinates (u, v) identify a location in the image for a point on the geometry. This parameterization is established when the shape is constructed. Second, the image is sampled at those coordinates, including wrapping and interpolation between **texels** (texture element.) Sampling occurs repeatedly during rendering and must work for coordinates both inside and outside the unit square.

The primary objectives of this assignment are to:

- Implement bilinear texture sampling with correct wrapping, including negative coordinates;
- Generate texture coordinates for spheres, cubes, and the teapot;
- Handle texture seams so that interpolation across a triangle does not span most of the image;
- Calculate light direction, distance, and attenuation for directional and point lights;
- Implement ambient, diffuse, and specular Phong contributions consistent with the viewport shader; and
- Create and save a textured scene that demonstrates the completed functionality.

Before beginning, review **include/TexCoords.h**, **include/Lighting.h**, **src/TexCoords.cpp**, and **src/Lighting.cpp**. The headers define the function contracts. The implementation notes in this handout explain the expected behavior and the reasons behind the requirements.

2 Building the Editor and Using Solution Libraries (Revisited)

As you work through the assignments, you will sometimes encounter functionality that was introduced in an earlier assignment but that you have not yet completed. Rather than requiring you to stop working until that functionality is finished, the editor supports **solution libraries**.

A solution library is a precompiled version of one part of the editor. You can add a supplied library to your project and tell the build system to use that completed implementation instead of your unfinished implementation.

This allows you to continue working on the current assignment while still using functionality from a previous assignment. This section is similar, but not exactly, the same as Assignment 2 and 3 and is included just for your reference.

2.1 Building the Editor

From the root directory of the editor project, extract the project if necessary and build it with the following commands:

```
tar xvzf editor.tar.gz
cd editor
make
```

If the build succeeds, an executable named editor will be created. Run the executable from the project directory:

```
./editor
```

You should run the editor from the project directory because the application loads resources such as shaders using paths relative to the current working directory.

2.2 Cleaning and Rebuilding

When you make changes to your source code, make will normally rebuild only the files that need to be recompiled.

If you want to remove the compiled object files and rebuild the editor from scratch, use:

```
make clean
```

Then rebuild the editor:

```
make
```

You can also combine the commands:

```
make clean && make
```

You should use make clean whenever you change which implementation of a topic is being used. This is especially important when switching between your implementation and a supplied solution library.

2.3 Using a Solution Library

Some functionality is implemented in later and earlier assignments. To allow you to use that functionality while working on your current assignment, solution libraries may be provided for your development environment.

Solution libraries are precompiled `.a` files. **Because compiled libraries are specific to the operating system, CPU architecture, and compiler/toolchain, you must use the library provided for your environment.** Do not copy a library from a different environment.

For example, if you want to use the completed camera implementation, you may be provided with:

```
libcamera.a
```

Step 1: Obtain the Library for Your Environment

Your instructor will provide the appropriate solution library separately based on your development environment. For example, the library provided for Linux may be different from the library provided for Windows or macOS.

Make sure you have the correct `.a` file for your environment before continuing.

Step 2: Copy the Library into `lib/solution/`

From the root directory of your editor project, copy the provided `.a` file into the `lib/solution/` directory.

Your directory should look similar to:

```
editor/
├──── lib/
│     └──── solution/
│           └──── libcamera.a
├──── src/
├──── Makefile
└──── ...
```

On Linux or macOS, you can copy the file using:

```
cp /path/to/libcamera.a lib/solution/
```

For example, if `libcamera.a` is in your Downloads directory:

```
cp ~/Downloads/libcamera.a lib/solution/
```

On Windows using MSYS2/MinGW, you can use:

```
cp /path/to/libcamera.a lib/solution/
```

or copy the file using Windows File Explorer and place it directly in:

```
editor\lib\solution\
```

Do not add the `.a` file manually to the `Makefile` or to `LIBS`. The build system will automatically locate and link the appropriate solution library when you use the corresponding build option.

Step 3: Build Using the Solution Library

Solution libraries are selected using a Make variable corresponding to the functionality they provide.

For example, to use the camera solution:

```
make clean
make CAMERA=solution
```

Then run the editor:

```
./editor
```

The CAMERA=solution option tells the build system to use the provided libcamera.a implementation instead of the camera implementation in your source tree.

For example, the following command:

```
make TRANSFORMS=solution
```

would use the provided transformation solution library instead.

The available solution libraries and their corresponding build options will be identified in the assignment instructions.

Important: Solution Libraries Are Environment-Specific

The source code for the assignment is shared across development environments, but the compiled `.a` libraries are not necessarily interchangeable.

For example:

```
Linux library       → Use with Linux
Windows library     → Use with Windows/MSYS2
macOS ARM library   → Use with Apple Silicon macOS
macOS Intel library → Use with Intel macOS
```

Always use the library supplied for your operating system, CPU architecture, and development environment. If you use an incompatible library, the linker may report errors or the resulting program may not run.

You do **not** need to modify the source code or Makefile to use a solution library. Copy the appropriate `.a` file into `lib/solution/` and use the corresponding `TOPIC=solution` build option.

Available Solution Libraries

The editor supports several optional solution libraries. These libraries allow you to temporarily use an instructor-provided implementation of a topic that you have not completed yet, so that you can continue working on later functionality.

For example, camera controls are part of a later assignment. While working on Assignment 2, you may use the provided camera solution library so that you can move around the scene while testing your transformations and geometry.

The editor currently recognizes the following solution libraries:

Functionality	Build Option	Library File
Model Transformations	TRANSFORMS=solution	libtransforms.a
Procedural Geometry and Normals	GEOMETRY=solution	libgeometry.a
OBJ Parsing	OBJPARSER=solution	libobjparser.a
Orbit, Pan, and Zoom Camera Controls	CAMERA=solution	libcamera.a
VR Camera / View Composition	VRCAMERA=solution	libvrcamera.a
Curves and Surfaces	CURVES=solution	libcurves.a

Functionality	Build Option	Library File
Skeletal Skinning	SKINNING=solution	libskinning.a
Keyframe Animation	ANIMATION=solution	libanimation.a
Texture Coordinates and Sampling	TEXTURES=solution	libtextures.a
Phong Lighting	LIGHTING=solution	liblighting.a
Ray-Tracer Shading	RAYTRACER=solution	libraytracer.a
Ray Intersections and BVH	BVH=solution	libbvh.a
Particle Integration and Forces	PARTICLES=solution	libparticles.a

Important: You should only use solution libraries that have been provided by your instructor. Some of the libraries listed above correspond to future assignments and may not be distributed until later in the semester.

2.4 Building the Solution Binary for Assignment 5

Assignment 5's solution file should include your code from Assignment 2, 3 and 4 along with the solution files for Assignment 5. To build a solution binary from scratch, use the following libraries:

```
lib/solution/libtransforms.a
lib/solution/libgeometry.a
lib/solution/libobjparser.a
lib/solution/libcamera.a
lib/solution/libanimation.a
lib/solution/libtextures.a
lib/solution/liblighting.a
```

If you want to build a solution binary with your existing code, all you will need are the following libraries:

```
lib/solution/libtextures.a
lib/solution/liblighting.a
```

Then clean your previous build:

```
make clean
```

Build the editor using the texture and lighting solutions:

```
make TEXTURES=solution LIGHTING=solution
```

To build the binary with Assignment 2, 3 and 4 solutions as well:

```
make GEOMETRY=solution TRANSFORMS=solution CAMERA=solution
OBJPARSER=solution ANIMATION=solution TEXTURES=solution
LIGHTING=solution
```

Run the editor:

```
./editor
```

To work on Assignment 5, just include the solutions from Assignments 2, 3, and 4 or use your complete code from the completed assignments. The editor will now use the previous completed solution while the rest of the application continues to use your own source code, or your other solutions files specified.

This allows you to work on Assignment 5 while using all the functionalities of the previous assignments.

2.5 Returning to Your Own Implementation

Solution libraries are optional. If you want to return to your own implementation, remove the solution option from the build command.

For example, after using:

```
make ANIMATION=solution
```

return to your own camera implementation with:

```
make clean
make
```

The normal make command uses the source code in your project rather than the animation solution library.

2.6 Multiple Solution Libraries

You can use more than one solution library at the same time.

For example:

```
make clean
make TRANSFORMS=solution GEOMETRY=solution
```

This tells the build system to use the supplied implementations for both transformations and geometry while continuing to compile all other topics from your source code.

Solution libraries can therefore be mixed with your own implementations. You do not have to replace the entire editor with a solution version.

For example:

```
make clean
make CAMERA=solution
```

means:

Camera	→	supplied solution library
Everything else	→	your source code

Similarly:

```
make clean
make TRANSFORMS=solution CAMERA=solution
```

means:

Transformations	→	supplied solution library
Camera	→	supplied solution library
Everything else	→	your source code

2.7 Important: Clean Before Switching Implementations

Always clean the project before changing a topic between your implementation and a solution library:

```
make clean
```

For example, if you first build with:

```
make CAMERA=solution
```

and later want to use your own implementation, do:

```
make clean
make
```

Likewise, if you are using your own implementation and want to switch to the solution:

```
make clean
make CAMERA=solution
```

Cleaning is important because previously compiled object files can remain in the project and interfere with the new build configuration.

2.8 Building a Full Solution Reference Editor

If you have been provided with all available solution libraries, you may build a fully functional version of the editor to use as a **reference while completing your assignments**.

This reference build can be useful when you want to compare your implementation against the completed behavior of the editor. For example, you can use it to check how a shape should render, how a control should respond, or how a later feature is expected to behave.

First, make sure all supplied `.a` files are located in:

```
lib/solution/
```

Then clean the project:

```
make clean
```

Build the editor using all available solution libraries:

```
make \
  TRANSFORMS=solution \
  GEOMETRY=solution \
  OBJPARSER=solution \
  CAMERA=solution \
  VRCAMERA=solution \
  CURVES=solution \
  SKINNING=solution \
  ANIMATION=solution \
  TEXTURES=solution \
  LIGHTING=solution \
  RAYTRACER=solution \
  BVH=solution \
  PARTICLES=solution
```

Then run:

```
./editor
```

This creates a version of the editor that uses the supplied solution implementation for every supported topic. The build system currently recognizes 13 solution topics.

You may use this full solution build as a **visual and behavioral reference** while developing your own code. For example, if you are unsure how your Cube, Sphere, transformations, camera controls, or another feature should behave, you can run the reference build and compare it against your own implementation.

The reference build does **not** replace the work required for the assignment. You are still responsible for implementing and submitting the source code required for your current assignment. Do not submit the solution libraries as your own implementation.

Also make sure that all solution libraries match your development environment. The `.a` files are specific to the operating system, CPU architecture, and compiler/toolchain being used.

2.9 Solution Libraries Do Not Replace Your Source Code

Using a solution library does **not** mean that you should copy someone else's source code into your project. The library is compiled code that is linked into your executable during the build process. Your source files remain in your project and can continue to be modified normally.

For example, when you build with:

```
make TRANSFORMS=solution GEOMETRY=solution
```

you can still edit and test your Assignment 5 files such as:

```
TexCoords.cpp
Lighting.cpp
```

The previous assignment's functionality is simply being supplied by the library while you work on the rest of the assignment.

2.10 Summary

The most common commands you will use are:

```
# Normal build
make

# Clean compiled files
make clean

# Clean and rebuild
make clean && make

# Build using the camera solution
make clean
make CAMERA=solution

# Build using the animation solution
make clean
make ANIMATION=solution

# Build using multiple solutions
make clean
make TRANSFORMS=solution GEOMETRY=solution
```

Remember: Solution libraries allow you to use completed functionality without replacing your own source code. They are provided so that you can continue making progress when a previous or future assignment component is not yet implemented.

3. Assignment Overview

The assignment combines two parts of the rendering process that are often treated separately. Texture coordinates connect a surface to an image, and the sampler determines the image color at a given coordinate. Lighting then combines the material properties with the properties of each light to calculate the final contribution.

3.1 Functionality

You will implement six functions across two source files. The texture-coordinate functions establish the mapping between geometry and images. The lighting functions calculate the geometric relationship between a surface point and a light, then evaluate the Phong terms for that light.

3.2 Defined Scope

The files you will be working on, and the section in those files that you will be working on are as follows:

Source file	Function	Responsibility
src/TexCoords.cpp	Texture::sample()	Sample an image with bilinear interpolation and wrapping outside [0, 1].
src/TexCoords.cpp	TexCoords::sphere()	Generate one UV coordinate for each sphere vertex.
src/TexCoords.cpp	TexCoords::cube()	Generate one UV coordinate for each triangle corner of the cube.
src/TexCoords.cpp	TexCoords::cylindricalPerCorner()	Generate cylindrical UV coordinates for the teapot and repair the wrap seam.
src/Lighting.cpp	lightGeometry()	Calculate light direction, distance, and attenuation for one light.
src/Lighting.cpp	phongContribution()	Calculate ambient, diffuse, and specular contributions for one light.
Source file	Function	Responsibility

Keep the existing public interfaces and data structures. Read **include/TexCoords.h** and **include/Lighting.h** before changing any implementation. The function declarations and comments in those headers are the specifications; this handout provides additional explanation.

3.3 Provided Components

The following components are already implemented and should be used as references rather than rewritten:

Component	Provided functionality
src/TextureCache.cpp	Decodes PNG and JPG images using stb_image, uploads them to OpenGL, and avoids duplicate loads by path.
TexCoords::planarPerFace	Provides planar projection for flat-faced meshes such as the Pyramid and Icosahedron in src/TexCoordsGiven.cpp.
TexCoords::parametricGrid	Provides UV coordinates for geometry generated through nested (i, j) loops, including the Torus, Mobius, and Custom shapes.
Shape::uploadTexCoords()	Uploads texture coordinates through the second VBO, using vertex attribute 3.
Material panel	Provides the texture picker and controls for ambient, diffuse, specular, and shininess coefficients.
shaders/fragment_shader.glsl	Implements viewport lighting and serves as the reference for the C++ lighting implementation.
Scene save/load	Persists the assigned texture using the texture= key.
data/textures/	

**The full contract for every one of them is written out in `include/Keyframe.h`. Read that header before you start. It is the specification; this handout is the explanation.

3.4 Where You Are Starting From

Build and run the editor. Insert a *Sphere*, open the Material panel, and assign **data/textures/uv-grid.png** to the shape. The sphere will not display the texture. Next, insert a *Pyramid* and assign the same image. The *Pyramid* should display the texture immediately.

This difference is expected. The Pyramid receives its UV coordinates from the provided **planarPerFace** implementation. The Sphere depends on `TexCoords::sphere()`, which currently returns an empty vector.

The shader only enables texturing when a shape has both a texture path and texture coordinates. A texture assigned to a shape without UV coordinates is therefore not drawn.

The same issue applies to the Cube and Teapot until their required texture-coordinate functions are implemented. Lighting begins from a different checkpoint: the existing **lightGeometry()** stub uses a fixed direction without falloff, while **phongContribution()** returns only the ambient term. These are simplified implementations that allow the project to build, but they do not provide the required lighting behavior.

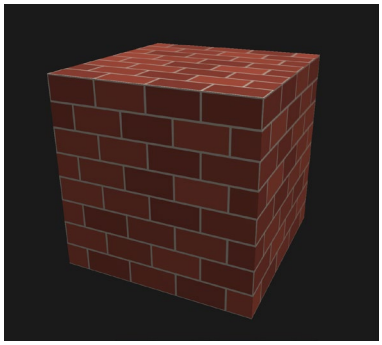
3.5 Texture Sampling and the Viewport

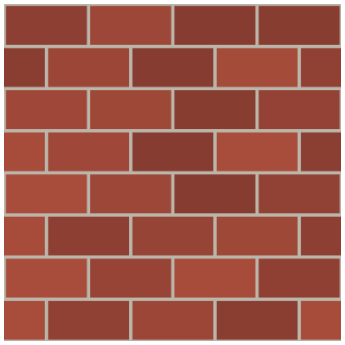
`Texture::sample()` is used by the ray tracer, which runs on the CPU. The editor viewport samples textures through OpenGL, which performs bilinear filtering in hardware and does not call your `Texture::sample()` implementation.

As a result, correct UV coordinates can appear in the viewport even when your sampler is incomplete. Conversely, an incorrect `Texture::sample()` implementation may have no visible effect in the editor until the ray tracer is implemented in Assignment 6. This does not reduce the importance of the sampler: it is a required part of this assignment and is covered by unit tests. Use the tests to evaluate the sampler rather than relying on the viewport.

4 Part 1 – Texture Sampling

Implement `Texture::sample(u, v)` to return the image color at the supplied texture coordinates. The function must handle coordinates outside the unit square, locate the surrounding texels, and blend their colors according to their distance from the sample point.





4.1 Wrapping Texture Coordinates

Texture coordinates may fall outside [0, 1]. For example, a texture repeated four times across a surface may use u values from 0 to 4. The seam repair required in Section 6 may also produce coordinates greater than 1.0. Wrap both coordinates into the interval [0, 1) rather than clamping them to the nearest edge.

Pay particular attention to negative values. In C++, `fmod(-0.375, 1.0)` returns -0.375 rather than 0.625. A wrapping implementation that handles only positive values may pass simple tests but produce a visible discontinuity along one edge of a repeated texture.

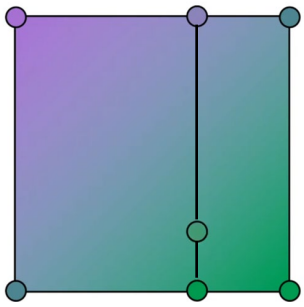
4.2 Accounting for the Half-Texel Offset

Texel coordinates refer to texel centers, not to the image boundaries. For an image dimension of size N, the center of texel i is located at (i + 0.5) / N. Your sampling calculation must account for this half-texel offset when converting normalized coordinates to image-space coordinates.

Omitting the offset shifts the sampled image by half a texel. The shift may be difficult to notice in a large photograph, but it is obvious in a small checkerboard image. The unit tests use small images to make this error detectable.

4.3 Bilinear Interpolation

The sampler must use bilinear interpolation rather than nearest-neighbor sampling. Identify the four texels surrounding the sample location and blend their values according to the horizontal and vertical distances from the sample point.



At the center of a texel, nearest-neighbor and bilinear sampling return the same color. A test suite that samples only at texel center therefore cannot reliably distinguish the two methods. The assignment tests sample between texel center: halfway between two texels, a correct result blends them equally; one-quarter of the way across, the result uses the corresponding 75/25 weighting. Simply averaging neighboring texels will not satisfy both cases.

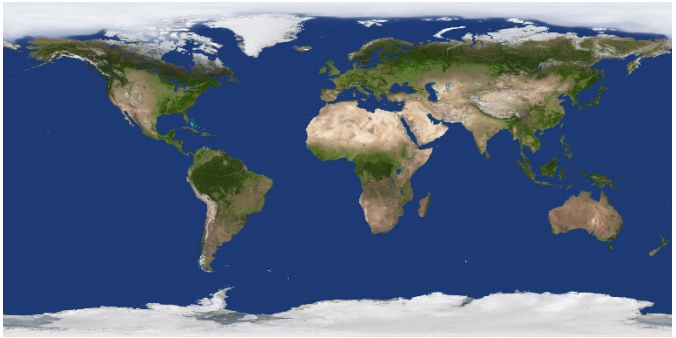
4.4 Image Row Orientation

Row 0 is at the bottom of the image. **TextureCache** flips images when loading them so that the image convention matches OpenGL. Therefore, v = 0 refers to the bottom of the picture. Keep this orientation consistent when converting normalized coordinates into image rows.

5 Part 2 – Texture Coordinate Parameterization

Texture coordinates establish which part of an image is assigned to each point on a surface. The required functions use different strategies because the underlying geometry is generated in different ways. The Sphere already has a natural parameterization, the Cube requires separate coordinates for each triangle corner, and the Teapot requires a cylindrical projection.





5.1 Sphere Parameterization

Implement `TexCoords::sphere(latSegments, lonSegments)` to generate one UV coordinate for each unique sphere vertex. The Sphere is the most direct case because `Sphere::setupSphere()` already generates its vertices from two angular parameters. Those angles, when normalized, provide the texture coordinates.

Use the same loop bounds and iteration order as the sphere-generation code so that each UV entry corresponds to the vertex at the same index. The sphere-generation loop begins at the north pole, while image row 0 is at the bottom. Flip the vertical coordinate so that the image orientation is correct.

5.2 Cube Parameterization

Implement `TexCoords::cube(vertices, faces, faceNormals)` to return one UV coordinate for each triangle corner. The output must contain three UV coordinates per face, ordered to match the corners in faces. Do not return only one coordinate per unique vertex.

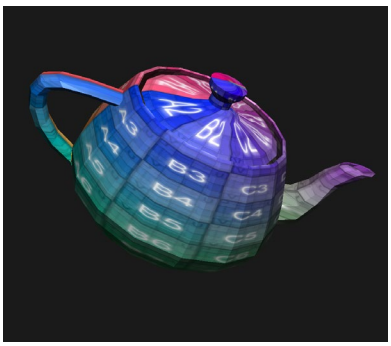
A cube has eight geometric corners, but each corner belongs to three faces. The same physical corner may need to appear at different locations in the texture on different faces. For example, a corner can be the top-left of one face and the bottom-right of another. A single UV coordinate stored per vertex cannot represent these distinct face mappings.

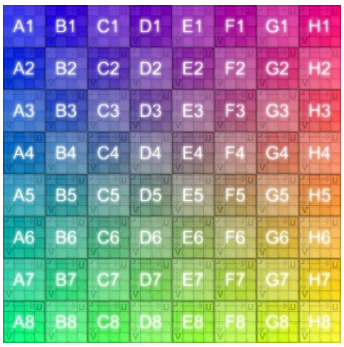
For each face, project the vertex onto the two coordinate axes that are not aligned with the face normal. The face normal is selected using **faceNormals[i / 2]**, because the cube has six face normals and twelve triangles, with two triangles per face.

Review **planarPerFace** in `src/TexCoordsGiven.cpp`. It provides a more general implementation of planar projection and is an appropriate reference. Make sure you understand how the dominant face-normal axis determines the remaining two projection axes; the **README** asks you to explain this relationship for the Cube.

6 Part 3 – Cylindrical Mapping and the Texture Seam

Implement `TexCoords::cylindricalPerCorner(vertices, faces)` to texture the Teapot. Unlike the Sphere and other parametric shapes, the Teapot vertices are provided as a triangle soup generated from a table of Bezier patches. There is no shared generation loop or pair of angular parameters from which the UV coordinates can be recovered.





6.1 Cylindrical Projection

Use the angle around the Y axis to calculate the horizontal texture coordinate. The angle is obtained from **atan2(z, x)** and remapped from the range $-\pi$ to π into the normalized u range. Calculate v by normalizing the vertex height using the minimum and maximum Y values of the mesh. Return three UV coordinates per face, in the same corner order used by the faces.

6.2 Understanding the Wrap Seam

A cylindrical projection has a discontinuity where the angle wraps from one end of its range to the other. Consider a triangle near the back of the Teapot with corner coordinates u = 0.98 and u = 0.02. These points are close together on the surface, separated by only 0.04 of a full turn. However, ordinary interpolation between the two values travels across almost the entire texture range.

The result is a stretched and mirrored portion of the image across the triangle. In the viewport, the error appears as a bright vertical scar along the back of the Teapot.

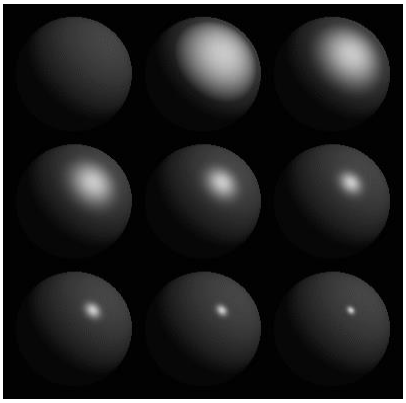
6.3 Repairing the Seam

Detect triangles whose u-coordinate range exceeds half of the texture range. For those triangles, adjust the low u coordinates by adding 1.0 so that the values lie on the same side of the discontinuity and the interpolated span is short. The sampler's wrapping operation then maps those adjusted values back into the image range.

This repair must be performed per triangle corner. After the adjustment, the same geometric vertex may have one u value when referenced by one triangle and a different u value when referenced by another. That is necessary to represent the seam correctly. The grader checks the resulting property: no triangle should span more than half the u range. Any implementation that satisfies this property is acceptable.

7 Part 4 – Phong Lighting

Implement **lightGeometry()** and **phongContribution()** in `src/Lighting.cpp`. These functions calculate the geometric relationship between a surface point and a light, then combine the light and material properties using the Phong reflection model. The ambient, diffuse, specular, and shininess values correspond to the controls in the **Material** panel.



7.1 Phong Illumination Model

The Phong illumination model approximates the interaction between light and a surface by combining three lighting components: ambient, diffuse, and specular illumination. Ambient illumination provides a constant contribution to the surface, diffuse illumination represents light scattered across the surface, and specular illumination produces highlights based on the relationship between the light source and the viewer. The combined illumination is calculated using the following equation:

$$I_{\text{phong}} = k_a I_a + k_d I_l \max(0, \mathbf{N} \cdot \mathbf{L}) + k_s I_l \max(0, \mathbf{R} \cdot \mathbf{V})^q$$

(k_a) Ambient reflection coefficient
(k_d) Diffuse reflection coefficient
(k_s) Specular reflection coefficient
(I_a) Ambient light intensity
(I_l) Light source intensity
($\mathbf{N}$) Normalized surface normal
($\mathbf{L}$) Normalized direction from the surface toward the light source.
($\mathbf{R}$) Normalized reflection direction of the incoming light.
($\mathbf{V}$) Normalized direction from the surface toward the viewer.
(q) Specular shininess exponent.

The dot products determine how strongly the surface responds to the light and viewing directions. The maximum function prevents negative contributions when the surface is oriented away from the light or when the specular reflection is outside the viewer's direction. The shininess exponent controls the size and concentration of the specular highlight. A larger exponent produces a smaller, sharper highlight, while a smaller exponent produces a broader highlight.

7.2 Point-Light Attenuation

A point light emits illumination from a specific position in the scene. Unlike a directional light, the direction from a point light to a surface changes depending on the surface position. The intensity of a point light also decreases as the distance from the light source increases. For this assignment, point-light attenuation follows the inverse-square relationship, in which illumination decreases proportionally to the square of the distance. The resulting illumination from a point light is calculated as follows:

$$I_{\text{final}} = \frac{k_a I_a + k_d I_l \max(0, \mathbf{N} \cdot \mathbf{L}) + k_s I_l \max(0, \mathbf{R} \cdot \mathbf{V})^q}{d^2}$$

where (d) represents the distance between the point light and the surface. The ambient contribution remains independent of the distance, while the diffuse and specular contributions are attenuated.

Directional lights do not use distance-based attenuation because they represent light arriving from a consistent direction. Point lights, however, must calculate the direction and distance from the light source to the surface when determining their illumination contribution.

When implementing the lighting calculations, ensure that the surface normal, light direction, reflection direction, and viewing direction are normalized before evaluating their respective dot products.

7.3 Light Direction, Distance, and Attenuation

Implement **lightGeometry(light, point)** for both directional and point lights.

- Directional light (type == 1): Directional lights emit parallel rays, so their direction is the same at every surface point. Their distance is effectively infinite, and their attenuation is 1. The stored direction describes the direction in which light travels. Calculate the direction from the surface point toward the light.

- Point light: A point light has a position. Calculate the direction and distance from the surface point to that position. Attenuation is calculated using the constant, linear, and quadratic coefficients:

$$attenuation = \frac{1}{(constant + linear * d + quadratic * d^2)}$$

Guard the zero-distance case. Do not attempt to normalize a zero-length direction vector.

7.4 Ambient, Diffuse, and Specular Contributions

Implement **phongContribution(...)** to calculate the three Phong terms for one light. Each term combines the light color with the corresponding material coefficient, but the terms do not all use the same inputs.

- **Ambient**: Ambient illumination is not scaled by reach. The reach value represents the shadow term, while ambient light represents indirect illumination that is not traced through a path to this light. Applying reach to ambient would make shadowed regions artificially black.

- **Diffuse**: Use the Lambert term, clamped to zero: `max(dot(n, lightDir), 0)`. Without this clamp, a surface facing away from a light could subtract illumination contributed by other lights.

- **Specular**: Use the **Phong reflection model**, not **Blinn-Phong**. Reflect the light direction about the surface normal and compare the reflected direction with the view direction.

- **Base color**: Do not multiply the specular contribution by **baseColor**. A specular highlight represents reflected light and should remain the light's color. For example, a red surface can have a white highlight.

7.5 Matching the Viewport Shader

The editor renders scenes through two separate implementations. The viewport uses OpenGL and **shaders/fragment_shader.glsl**, while the C++ implementation in **Lighting.cpp** is used by the ray tracer. The GLSL shader is provided and serves as the reference for your C++ implementation.

The two implementations must agree term for term. If the C++ calculation differs from the GLSL calculation, the viewport and ray-traced image may show different lighting even though both images appear plausible. The

lighting tests calculate expected values using the same term order as the GLSL shader, so changes that cause the implementations to diverge will be detected.

7.6 Behaviors That Are Deliberate

- Specular is not gated by $\mathbf{N} \cdot \mathbf{L}$. A surface facing away from a light may still fall within the Phong reflection lobe and receive a specular contribution. This behavior is shared by the provided GLSL shader; do not change only the C++ implementation.

- Ambient is unshadowed. Do not multiply the ambient contribution by reach. This is intentional and is part of the required agreement with the shader.

8 Testing Your Assignment

8.1 Visual Texture Checks

Assign **data/textures/uv-grid.png** to the **Sphere**, **Cube**, and **Teapot** as you complete their parameterization functions. Inspect the orientation and placement of the grid on each shape. The grid makes stretching, incorrect orientation, and mapping discontinuities easier to identify than a photographic texture.
Use the **Pyramid** as a reference for the provided planar projection. Compare the **Sphere**, **Cube**, and **Teapot** against the expected mapping behavior described in Sections 5 and 6.

8.2 Testing Lighting

Use the **Material panel** to change **ambient**, **diffuse**, **specular**, and **shininess** values. Add both directional and point lights. Move a point light close enough to a shape that its distance attenuation is visible, and compare the resulting highlights and diffuse shading as you adjust the material coefficients.

Compare the viewport lighting against the behavior specified in **shaders/fragment_shader.glsl**. In particular, check that ambient illumination remains present in shadowed regions and that specular highlights are not tinted by the surface base color.

8.3 Suggested Implementation Order

The following order allows you to see progress early while keeping the CPU sampler and lighting tests independent from the UV work:

`TexCoords::sphere()` - the sphere's vertex-generation loop already contains the angular parameters needed for its UV coordinates.

`TexCoords::cube()` - implement per-corner coordinates and understand why a cube cannot use one UV per unique vertex.

`Texture::sample()` - implement wrapping and bilinear sampling. This function is unit-tested but does not visibly affect the editor viewport yet.

`TexCoords::cylindricalPerCorner()` - implement the Teapot projection and repair its wrap seam.

`Lighting.cpp` - implement **lightGeometry()** and **phongContribution()**. These functions are independent of the UV functions.

The repository also contains **./texture_scene**, which renders a textured scene to a **.ppm** file through the ray tracer. The ray tracer is the subject of Assignment 6 and is still stubbed at this stage. Consequently, **texture_scene** will currently produce a flat-color result and should be revisited after Assignment 6.

9 Create and Save Your Textured Scene

Once the texture and lighting features are working, create a scene that demonstrates the different parameterizations, material properties, and light types. Arrange the objects and lights so that the effects are easy to see. Your scene should demonstrate the completed functionality rather than simply contain the required objects.

9.1 Scene Requirements

Your saved scene must include all of the following:

- A Sphere, a Cube, and the Teapot, each with a different texture assigned.
- At least one shape with non-default material coefficients. Adjust shininess so that the difference between a low-shininess and high-shininess material is visible.
- At least two lights, including at least one point light positioned close enough to a surface for its attenuation to be apparent.

Choose textures that make the UV mapping easy to inspect. A grid or checker pattern can reveal stretching and orientation errors, while a more detailed image can demonstrate the appearance of a completed material. Arrange the lights so that diffuse shading, specular highlights, and point-light falloff can be observed.

9.2 Save and Verify Your Scene

Save the completed scene through **File > Save Scene** as **textures.scene**. Reopen the saved file in your final build and verify that the three shapes retain their assigned textures, the material coefficients are preserved, and the lights reproduce the intended appearance.

If your scene depends on external assets, verify that the paths remain valid when the project is moved to a different folder. Include any required external assets with your submission.

10 Submission

Submit one compressed archive containing your implementation, saved scene, README, and any supporting files required to build and run the project. Do not submit loose files.

10.1 Source Code

Complete the six required functions in `src/TexCoords.cpp` and `src/Lighting.cpp`. Set your windows title in **File > Options** to your name.

Include your source code and the project files needed to build the editor. Your final build must use your own implementations of `Texture::sample(), TexCoords::sphere(), TexCoords::cube(), TexCoords::cylindricalPerCorner(), lightGeometry(), and phongContribution()`.

10.2 README

Create a README.txt or README.md file containing the following information:

- Your name and anyone with whom you collaborated, including a description of the collaboration.
- References, books, websites, or other resources you found useful.
- An explanation of why cube() returns one UV coordinate per triangle corner rather than one per vertex. Explain your answer in terms of the Cube's faces and shared corners.
- A description of what the Teapot's wrap seam looks like before it is repaired, and why the repair cannot be represented using one UV coordinate per vertex.
- An explanation of why nearest-neighbor sampling may be difficult to notice on a photograph such as **2k_earth_daymap.jpg** but obvious on checker.png. Explain why the tests sample between texel centers.
- Known problems with your implementation, how to reproduce them, and what you believe is causing them. Be specific.
- Any additional comments about the assignment.

10.3 Submission Checklist

Your archive must contain:

- Your source code and the project files needed to build it.
- Your compiled editor executable for your development environment.
- *textures.scene*.
- Any external assets required to load the scene.
- Your README.txt or README.md.

Use ZIP or TAR.GZ format and submit one archive. Before submitting, extract the archive into a fresh folder, follow your build instructions, open textures.scene, and verify that the scene loads with all required textures, materials, and lights. This final check should confirm that the archive contains everything needed to compile, run, and review your work.